

SISTEMAS DISTRIBUÍDOS

Aula 20 — Algoritmos Distribuídos III: Replicação e Consistência de Dados

6º Semestre CC/ES-SI · Prof. Leandro Borges · 22/10

Agenda da Aula

1 Por que replicar dados: disponibilidade, desempenho e tolerância a falhas

2 Modelos de consistência: forte (strict) vs. eventual

3 Meio-termo: consistência causal e garantias orientadas à sessão

4 Estratégias de replicação: síncrona vs. assíncrona

5 Topologias: master-slave (primary-backup) vs. multi-master

6 O teorema CAP: por que não dá para ter as três ao mesmo tempo

7 CAP na prática: sistemas reais que escolhem CP ou AP

Por Que Replicar Dados?

Disponibilidade

Réplicas permitem que o sistema continue respondendo mesmo que uma máquina, um data center ou uma região inteira fique inacessível — os dados não dependem de um único ponto de falha.

Desempenho e latência

Manter cópias fisicamente próximas dos usuários reduz o tempo de resposta, e distribuir leituras entre várias réplicas evita sobrecarregar uma única máquina.

Tolerância a falhas

Se uma réplica falha, o sistema continua operando a partir das réplicas restantes, e a réplica perdida pode ser reconstruída a partir de uma cópia saudável.

Modelos de Consistência: Visão Geral

Consistência forte (strict/strong)

Toda leitura, em qualquer réplica, retorna o valor mais recente escrito — como se existisse uma única cópia dos dados. É o resultado mais previsível, mas exige coordenação entre réplicas a cada escrita, o que aumenta a latência e reduz a disponibilidade quando há partição de rede.

Consistência eventual

Escritas se propagam entre réplicas de forma assíncrona; se não houver novas escritas, todas as réplicas eventualmente convergem para o mesmo valor. Oferece baixa latência e alta disponibilidade, mas leituras podem retornar dados desatualizados (stale) por um curto período.

Meio-Termo: Consistência Causal e Garantias de Sessão

Consistência causal

Garante que operações causalmente relacionadas (uma escrita que depende de uma leitura anterior) sejam vistas por todos os processos na mesma ordem; operações concorrentes, sem relação de causa e efeito, podem ser vistas em ordens diferentes por réplicas diferentes. Preserva a relação “causa antes do efeito” (ex.: um comentário sempre aparece depois da postagem que ele responde) sem pagar o custo total da consistência forte.

Garantias orientadas à sessão (read-your-writes, monotonic reads)

Garantias mais pontuais, populares em sistemas de consistência eventual para amenizar a experiência do usuário: o próprio cliente sempre enxerga as escritas que ele mesmo fez, e leituras sucessivas de um mesmo cliente nunca “andam para trás” no tempo, mesmo que outras réplicas ainda estejam desatualizadas.

Modelos de Consistência: Comparativo

Modelo	Garantia oferecida	Latência / desempenho	Exemplo de uso
Forte (strict)	Toda leitura reflete a escrita mais recente, em qualquer réplica	Alta latência: coordenação a cada escrita	Bancos relacionais com replicação síncrona; sistemas bancários
Causal	Operações relacionadas por causa-efeito respeitam a ordem; concorrentes podem divergir	Latência intermediária	Redes sociais (feeds, comentários); ex. COPS
Eventual	Réplicas convergem “eventualmente”, sem garantia de ordem imediata	Baixa latência, alta disponibilidade	DNS; caches distribuídos; DynamoDB, Cassandra

Replicação Síncrona vs. Assíncrona

Síncrona

O coordenador só confirma a escrita ao cliente depois que a(s) réplica(s) também confirmarem que gravaram o dado. Garante forte durabilidade e consistência, mas a latência da escrita é limitada pela réplica mais lenta, e uma réplica indisponível pode travar novas escritas.

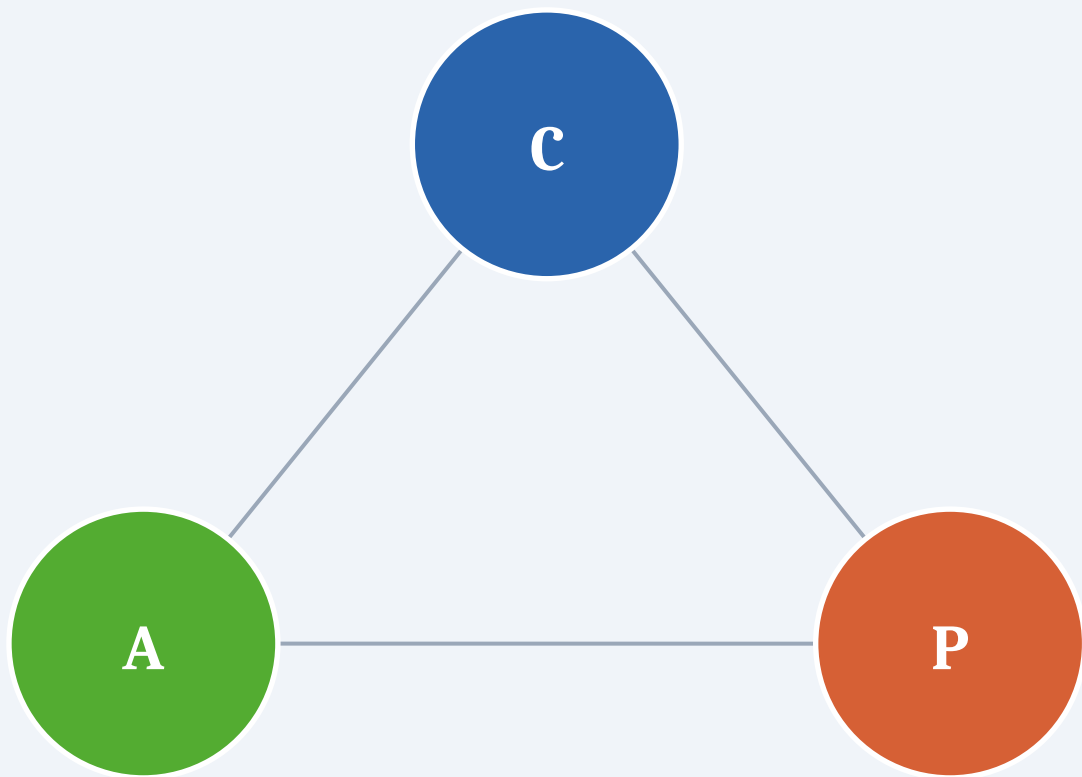
Assíncrona

O coordenador confirma a escrita ao cliente assim que grava localmente, e propaga a atualização às réplicas em segundo plano. Escritas são rápidas e o sistema tolera réplicas lentas ou offline, mas existe uma janela de réplicas desatualizadas — e dados confirmados podem se perder se o nó principal falhar antes de propagar.

Topologias de Replicação: Master-Slave vs. Multi-Master

Aspecto	Master-slave (primary-backup)	Multi-master
Quem aceita escritas	Só o nó primário; secundários geralmente só leitura	Qualquer réplica pode aceitar escritas diretamente
Conflitos de escrita	Não ocorrem: só existe uma fonte de verdade por vez	Podem ocorrer escritas concorrentes na mesma chave
Ponto único de falha	Primário é gargalo de escrita e ponto único até o failover	Sem gargalo único; melhor disponibilidade de escrita
Resolução de conflito	Desnecessária — ordem de escrita já é única	Exige estratégia: last-write-wins, vetores de versão, CRDTs

O Teorema CAP



Escolha apenas 2 de 3

C — Consistência: toda leitura retorna a escrita mais recente, em qualquer nó.

A — Disponibilidade: toda requisição recebe resposta, mesmo que não seja a mais recente.

P — Tolerância a Partição: o sistema continua operando mesmo quando mensagens entre nós se perdem ou atrasam.

Partições de rede são inevitáveis em sistemas reais — P é praticamente obrigatório. A escolha de fato acontece entre C e A no momento em que a partição ocorre.

CAP na Prática: Escolhas CP vs. AP

CP — Consistência + Tolerância a Partição

Durante uma partição, o sistema prefere recusar ou atrasar respostas a arriscar retornar dados desatualizados. Exemplos: HBase, MongoDB (configuração padrão), Apache ZooKeeper, bancos relacionais com replicação síncrona.

AP — Disponibilidade + Tolerância a Partição

Durante uma partição, o sistema continua respondendo com o dado disponível localmente, mesmo que desatualizado, e reconcilia depois. Exemplos: Amazon DynamoDB, Apache Cassandra, Riak, CouchDB. Fora de uma partição (o caso comum), a maioria dos sistemas reais entrega tanto C quanto A — a escolha CAP só se manifesta no instante em que a partição de fato acontece.

Próxima Aula

Sistemas Tolerantes a Falhas I: Modelos de Falha e Detecção

27/10 — que tipos de falha um sistema distribuído precisa tolerar, e como detectá-las.

Referências desta Aula

- BREWER, E. A. Towards Robust Distributed Systems (CAP Theorem Keynote). ACM PODC, 2000.
- GILBERT, S.; LYNCH, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, v. 33, n. 2, 2002.
- KLEPPMANN, M. Designing Data-Intensive Applications. Sebastopol: O'Reilly Media, 2017.
- TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.